UAV Software Engineering: Quality Tooling

Testing, Static Analysis & Continuous Integration

1 Testing, Static Analysis & Continuous Integration

In safety-critical robotics, writing code is only a fraction of the engineering effort; mathematically and empirically proving that the software behaves deterministically under all conditions is the primary deliverable. This section details the quality gates, architectural patterns, and testing infrastructure required for modern UAV software.

1.1 The Robotics Testing Pyramid

The validation of flight software follows a strict testing pyramid, where each layer is designed to catch specific classes of failures. Relying solely on one layer leads to either blind spots or unsustainable execution times.

- **Unit Tests (L1):** Validate pure C++ algorithms (e.g., PID math, coordinate transformations, state machine transitions) in isolation. They run in microseconds and enforce algorithmic correctness, but inherently cannot catch system-level timing anomalies or memory bus contention.
- **Component Tests (L2):** Verify single ROS 2 nodes or PX4 modules (e.g., parameter loading, uORB publishing logic) using mocked middleware dependencies. This ensures the node respects its lifecycle contract (Unconfigured $\rightarrow$ Inactive $\rightarrow$ Active).
- **Integration Tests (L3):** Ensure multiple components (e.g., sensor fusion communicating with offboard control) interact correctly over the DDS middleware. They validate QoS compatibility and message serialization.
- **Simulation / SITL (L4):** Validates the full software stack running in lockstep against a physics engine (Gazebo). It catches control logic bugs, aerodynamic instabilities, and navigation faults. However, the "reality gap" means SITL cannot accurately simulate hardware vibrations, EMI (Electromagnetic Interference), or battery voltage sag under load.
- **Flight Tests (L5):** The apex of the pyramid. Validates the sim-to-real gap, hardware drivers, and real-world environmental disturbances. It is the most expensive and dangerous layer; a failure here implies a failure in the lower quality gates.

1.2 Testability and GoogleTest Framework

GoogleTest (gtest) and GoogleMock (gmock) provide the standard framework for C++ validation. A robust test suite moves beyond simple `EXPECT_EQ` assertions by leveraging advanced testing paradigms:

- **Test Fixtures (`TEST_F`):** Enforce strict setup and teardown phases, ensuring a pristine memory state and isolated environment for each test case, preventing state leakage between tests.
- **Parameterized Tests (`TEST_P`):** Execute the same algorithmic logic against arrays of edge-case inputs (e.g., testing a geofence algorithm with coordinates inside, outside, exactly on the boundary, and at the poles).
- **Death Tests (`EXPECT_DEATH`):** Verify that fatal assertions and aborts trigger correctly when the system is forced into an illegal state (e.g., a null pointer passed to a critical flight controller function).

1.2.1 Dependency Injection and the Clock Abstraction

A common anti-pattern in robotics is calling `std::chrono::steady_clock::now()` directly inside a control loop. This tightly couples the algorithm to the hardware clock, making it impossible to test timeout logic without actually halting the test thread for seconds. By injecting a virtual `Clock` interface, the test suite can manipulate time synthetically. The test can advance the mock clock by 500 ms instantly to verify that a "Link Loss" failsafe triggers exactly when expected, ensuring tests remain deterministic and execute in milliseconds.

1.3 Testing Concurrency and Floating-Point Mathematics

Concurrent data structures, such as the lock-free SPSC ring buffer, cannot be proven safe with standard sequential logic tests. They require *stress testing* with heavily contested producer and consumer threads. Furthermore, they must be compiled with **ThreadSanitizer (TSan)**. TSan maps application memory to "shadow memory" to track read/write operations per thread, mathematically proving the presence or absence of data races at the cost of a 5x-10x performance overhead.

For mathematical operations, exact equality (`==`) is forbidden due to IEEE 754 floating-point representation errors and precision loss during operations like trigonometric conversions. Tests must employ `EXPECT_NEAR` with a strictly defined epsilon, or `EXPECT_DOUBLE_EQ` which compares the binary representation to ensure they are within 4 ULP (Units in the Last Place).

1.4 Static Analysis and Code Formatting

Relying on the compiler to catch semantic errors is insufficient. A professional pipeline integrates continuous static analysis:

- **clang-tidy:** An AST-aware (Abstract Syntax Tree) linter that enforces modern C++ idioms and catches performance traps (e.g., `performance-unnecessary-value-param` for missed const-references). It is configured to enforce safety guidelines such as the C++ Core Guidelines and CERT C++ rules.
- **cppcheck:** Detects deeper control-flow anomalies, out-of-bounds array accesses, null pointer dereferences, and uninitialized variables without compiling the code.
- **clang-format:** Enforces deterministic codebase styling (e.g., Google or LLVM style), entirely eliminating formatting debates during Code Reviews and ensuring git diffs only reflect actual logic changes.
- **-Werror Policy:** Treating all compiler warnings (`-Wall -Wextra -Wpedantic`) as hard errors ensures that technical debt and ambiguous casts are never merged into the main branch.

1.5 Coverage Metrics and Their Limitations

Tools like `gcov` and `lcov` measure which lines and branches of code are executed during the test suite. While achieving 100% line coverage is often treated as a gold standard, it is an anti-pattern: it leads to "assertion-free testing" where developers write meaningless tests simply to hit lines of code.

A superior metric is **Branch Coverage** combined with MC/DC (Modified Condition/Decision Coverage) on critical paths. A targeted 85% branch coverage on the general codebase, with a strict 100% requirement on failsafe logic and state machine transitions, provides significantly higher assurance than blanket line coverage.

1.6 Continuous Integration (GitHub Actions)

Continuous Integration automates the execution of these quality gates on every commit. A standard GitHub Actions workflow for robotics utilizes a *Build Matrix* to compile and test the codebase against multiple configurations (e.g., GCC 11 vs Clang 14, ROS 2 Humble vs Jazzy) simultaneously.

- **Caching Strategies:** Compiling C++ templates and ROS 2 workspaces is CPU-intensive. Leveraging `ccache` and Docker layer caching drastically reduces pipeline execution time from tens of minutes to seconds.

- **Artifact Retention:** If an integration test or SITL scenario fails in CI, the workflow must automatically upload the resulting `.ulog` files, core dumps, and `rosbag` data as downloadable artifacts. This allows engineers to perform post-mortem analysis locally.
- **Pre-commit Hooks:** Formatting (`clang-format`) and fast linting are executed locally on the developer's machine via git hooks before the code reaches the CI server, saving compute resources and immediate feedback cycles.